

A Reading You Can Trust

A stale or impossible reading is worse than none — it lies with confidence. Guard the data: detect bad readings and fail safe.

ESP32-004 · ~40 MIN · BEGINNER

1. The one idea

A reading you can't trust is worse than no reading — it lies with a straight face. This lesson guards the *data*: detect a stale, frozen, or out-of-range sensor value and **fail safe** instead of acting on a number that isn't real.

2. Why it matters

Course 1 guarded an output; this guards an input. Sensors fail in quiet ways — a wire falls off and the value freezes, a sensor browns out and reads zero, a slow sensor returns the same stale number for minutes. A controller that trusts those numbers does exactly the wrong thing, confidently. Knowing *when not to believe a sensor* is the difference between a robust node and a fragile one.

3. Parts & wiring

The Course 3 bench — ESP32 + a sensor on I2C — plus the same OLED to show trust state.

- ESP32 dev board
- BME280 (or DHT22) on I2C (SDA 21 / SCL 22)
- SSD1306 OLED on the same bus

Same I2C wiring and the same one-pull-up-pair rule as Course 3. Mount the sensor a little away from the regulator and the OLED — their heat skews temperature, which is its own kind of bad reading.

4. Predict (do this before uploading)

- You unplug the sensor mid-run. What does a naïve `readTemperature()` return — an error, the last value, or NaN?
- A humidity reading of 250% — should the node display it, or reject it?
- If the value hasn't changed in 30 seconds, is the sensor calm, or dead?

5. Build it — three trust checks

A reading passes only if it is **present**, **in range**, and **fresh**:

```
float lastGood = NAN;
unsigned long lastChangeMs = 0;
const unsigned long STALE_MS = 10000;    // no change for 10 s = suspect

enum Trust { OK, MISSING, OUT_OF_RANGE, STALE };

Trust readTrusted(float &out) {
    float t = bme.readTemperature();

    if (isnan(t)) return MISSING;          // present?
    if (t < -40 || t > 85) return OUT_OF_RANGE;    // plausible? (BME280 spec)

    if (!isnan(lastGood) && t != lastGood) lastChangeMs = millis();
    lastGood = t;
    out = t;

    if (millis() - lastChangeMs > STALE_MS) return STALE; // fresh?
    return OK;
}

void loop() {
    float t;
    Trust s = readTrusted(t);
    oled.clearDisplay();
    oled.setCursor(0, 0);
    if (s == OK) oled.printf("Temp %.1f C OK", t);
    else        oled.printf("SENSOR FAULT\n%s",
                           s == MISSING ? "no data" : s == OUT_OF_RANGE ? "out of range" :
                           "stale");
    oled.display();
    delay(500);
}
```

The new idea: the value and your *confidence* in it travel together. The node never shows a fault reading as if it were real — it says **SENSOR FAULT** and names why.

6. Test against your prediction

Run it, then provoke each failure: pull the sensor's SDA wire (→ `no data`), warm it past 85°C in your imagination by faking the range (→ `out of range`), and let it sit perfectly still (→ `stale`). Each fault should be caught and named, not displayed as a temperature. Did the unplugged sensor behave the way you predicted in §4?

7. What goes wrong (pre-loaded debugging)

- **Faults never trigger** → you're reading a module that returns the last value forever; add the staleness check, don't rely on NaN alone.
- **Everything reads stale** → your "changed?" compare is exact-equal on a noisy float that never repeats; allow a small tolerance, or compare to a rounded value.
- **False out-of-range** → wrong sensor limits; use the part's real spec, not a guess.
- **Temperature reads high and never faults** → self-heating from the regulator/OLED; move the sensor, and treat a slow upward drift as a placement bug, not weather.

8. Checkpoint

The node now distinguishes a *good* reading from a *bad* one and refuses to act on the bad. Present, in-range, fresh — three cheap checks that turn a sensor from a thing you hope works into a thing you can trust or knowingly distrust. Every field instrument you build from here needs this guard on its inputs.

9. Push further

- Hold the last *good* value and act on it briefly during a glitch, then fail safe if the fault persists.
- Add a second sensor and cross-check them — disagreement is its own fault signal.
- Log fault events with timestamps — the first step toward the evidence trace in Course 8.